

Frequent pitfalls and how to avoid them

Context & memory

Pasting raw logs into the prompt Dumps the entire file to Claude instead of isolating the relevant part. The context fills up within a few exchanges.

Not using CLAUDE.md Repeating project conventions every session. Centralize in `CLAUDE.md` everything that needs to be constant.

Ignoring context warnings Use `/context` to inspect capacity and relevance. Compact or hand off when needed; no percentage guarantees a particular response quality.

Prompting

Vague requests `Refactor this file` with no context produces an unpredictable result. Be specific: what, why, constraints.

Iterating without checkpoints Chaining 10 exchanges without validating. Check the diff after each significant change.

Accepting without reviewing `--dangerously-skip-permissions` without understanding what Claude is doing. Always auto-accept within a limited scope.

Configuration

CLAUDE.md too long Putting all project documentation in `CLAUDE.md`. The startup context should be concise (< 200 lines).

Hooks without testing Writing a complex hook without testing it in isolation. Test manually first, then enable it.

Ignoring settings.local.json Not gitignoring this file lets personal config leak into the team repository.

Security

Unhandled prompt injection Letting Claude process external content (emails, issues) without guardrails. Use a restricted mode with a tool whitelist.

Secrets in context Passing API keys in the prompt or reading them via `@file`. Use environment variables instead.

--dangerously-skip-permissions without sandbox Running in full bypass on a shared machine or in production. Reserve for ephemeral environments only.

Golden rules

Give clear and concise instructions, then let Claude work. Come back to validate in small increments rather than letting everything happen at once. Document important decisions in `CLAUDE.md` for future sessions.